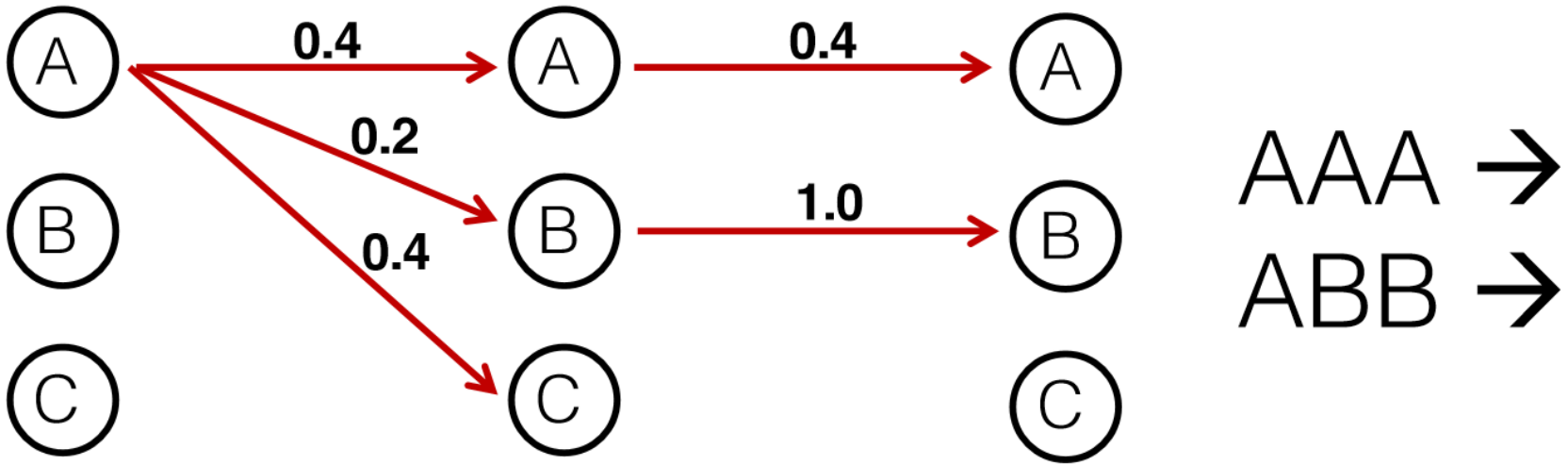
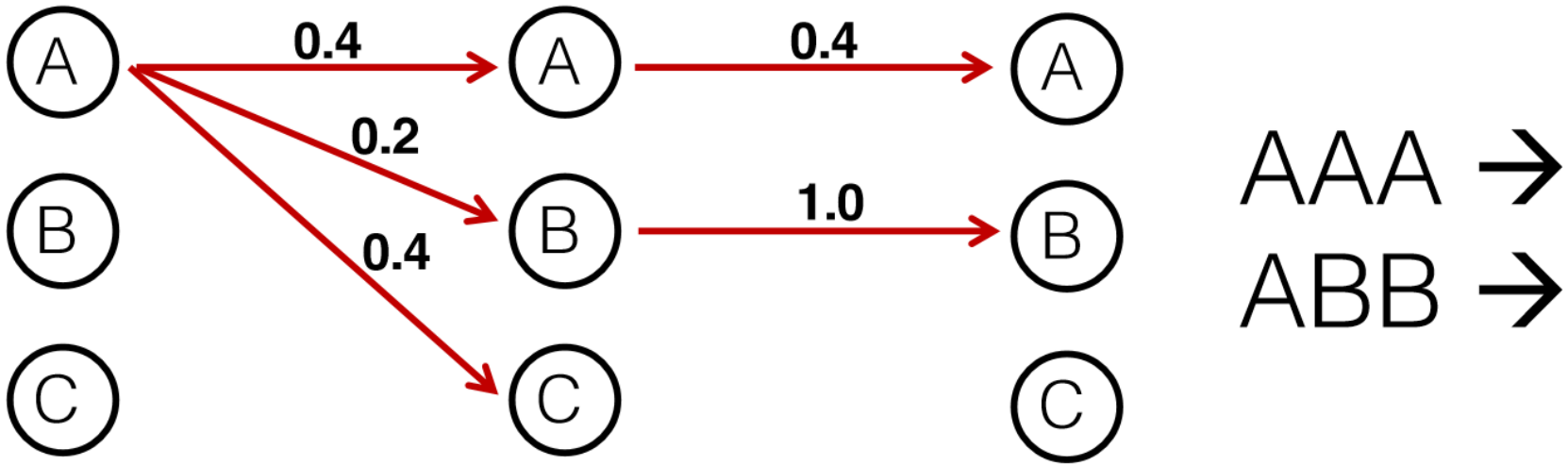
The Label Bias



- We have so far considered locally-normalized models
  - Namely, the scores (probability term) of each transition must sum up to 1
- This assumption may lead to a bias towards labels that have high-probability transitions, even if rarely observed

The Label Bias



- This happens in real-life settings
  - For instance, in NER, consider a label which is rare, but tends to have long names (e.g., the “continue” category of nobility names or books by E. Lockhart)

Books by E. Lockhart

The Boyfriend List: 15 Guys, 11 Shrink Appointments, 4 Ceramic Frogs and Me, Ruby Oliver (Ruby Oliver, #1)

The Boy Book: A Study of Habits and Behaviors, Plus Techniques for Taming Them (Ruby Oliver, #2)

The Treasure Map of Boys: Noel, Jackson, Finn, Hutch, Gideon—and me, Ruby Oliver (Ruby Oliver, #3)

Real Live Boyfriends: Yes. Boyfriends, Plural. If My Life Weren't Complicated, I Wouldn't Be Ruby Oliver

The Boyfriend Quartet: 15 Boys, 43 Lists, 120 Footnotes, and Too Many Panic

Attacks to Count, All in Four Novels about Ruby Oliver

Globally-normalized Models

- Newer, higher-powered discriminative sequence models
- Do not decompose training into independent local regions
  - Thus avoiding the label bias
- Often take a very long time to train - require repeated inference on training set

Sequence Conditional Random Fields (CRFs)

- CRFs are similar to MEMMs, but the normalization is global.
- This solves the label bias: the scores of the next state given the current one need not sum up to 1

$$P(y|x) = \frac{\prod_{j=1}^{n(i)} e^{f(j,y_j,y_{j-1},x)^T \cdot w}}{Z(x;w)}$$

$$Z(x;w) = \sum_{y} \prod_{j=1}^{n(i)} e^{f(j,y_j,y_{j-1},x)^T \cdot w}$$

Training in CRF

- Given training examples $(x^{(1)},x^{(2)},\ldots,x^{(N)})$ and labels $(y^{(1)},y^{(2)},\ldots,y^{(N)})$, we define the conditional log-likelihood:

$$Pr(y^{(1)},\ldots,y^{(N)}|x^{(1)},\ldots,x^{(N)}) = \prod_{i=1}^{N} Pr(y^{(i)}|x^{(i)}) = \prod_{i=1}^{N} \frac{\prod_{j=1}^{n(i)} exp(f(j,y_j^{(i)},y_{j-1}^{(i)},x^{(i)})^T \cdot w)}{Z(x^{(i)};w)}$$

$$Z(x^{(i)};w) = \sum_{y} \prod_{j=1}^{n} exp(f(j,y_j,y_{j-1},x^{(i)})^T \cdot w)$$

Estimating *w*

- We use maximum likelihood :

$$LL(w) = \sum_{i=1}^{N} \Big[\sum_{j=1}^{n(i)} f(j, y_j^{(i)}, y_{j-1}^{(i)}, x^{(i)})^T \cdot w - log(Z(x^{(i)}; w)) \Big]$$

- No close formula. We use gradient-based methods:

$$\frac{\partial LL}{\partial w_k} = \sum_{i=1}^{N} \Big[\sum_{j=1}^{n(i)} f_k(j, y_j^{(i)}, y_{j-1}^{(i)}, x^{(i)}) - \sum_{y \in T^{n(i)}} Pr(y|x^{(i)}) \cdot \sum_{j=1}^{n(i)} f_k(j, y_j, y_{j-1}, x^{(i)}) \Big]$$

CRFs: Gradient Computation

- The second term is more tricky to compute, as it sums over an exponential number of elements:

$$\sum_{y \in T^{n(i)}} Pr(y|x^{(i)}) \cdot \sum_{j=1}^{n(i)} f_k(j, y_j, y_{j-1}, x^{(i)}) =$$

$$\sum_{j=1}^{n(i)} \sum_{y \in T^{n(i)}} Pr(y|x^{(i)}) \cdot f_k(j, y_j, y_{j-1}, x^{(i)}) =$$

$$\sum_{j=1}^{n(i)} \sum_{y_j, y_{j-1}} \boxed{Pr(y_j, y_{j-1}|x^{(i)})} \cdot f_k(j, y_j, y_{j-1}, x^{(i)})$$

CRFs: Gradient Computation

- The marginals require dynamic programming to compute: (we should compute them for every pair of values for y_j and y_{j-1}

$$Pr(y_j, y_{j-1}|x^{(i)})$$

- We can solve this with dynamic programming

CRFs: Forward-Backward Algorithm

- Goal (for every *a,b*): $Pr(y_j = a, y_{j-1} = b|x^{(i)}) = \sum_{y \in T^{n(i)}: y_j = a, y_{j-1} = b} Pr(y|x^{(i)})$

Approach: dynamic programming

- Define:

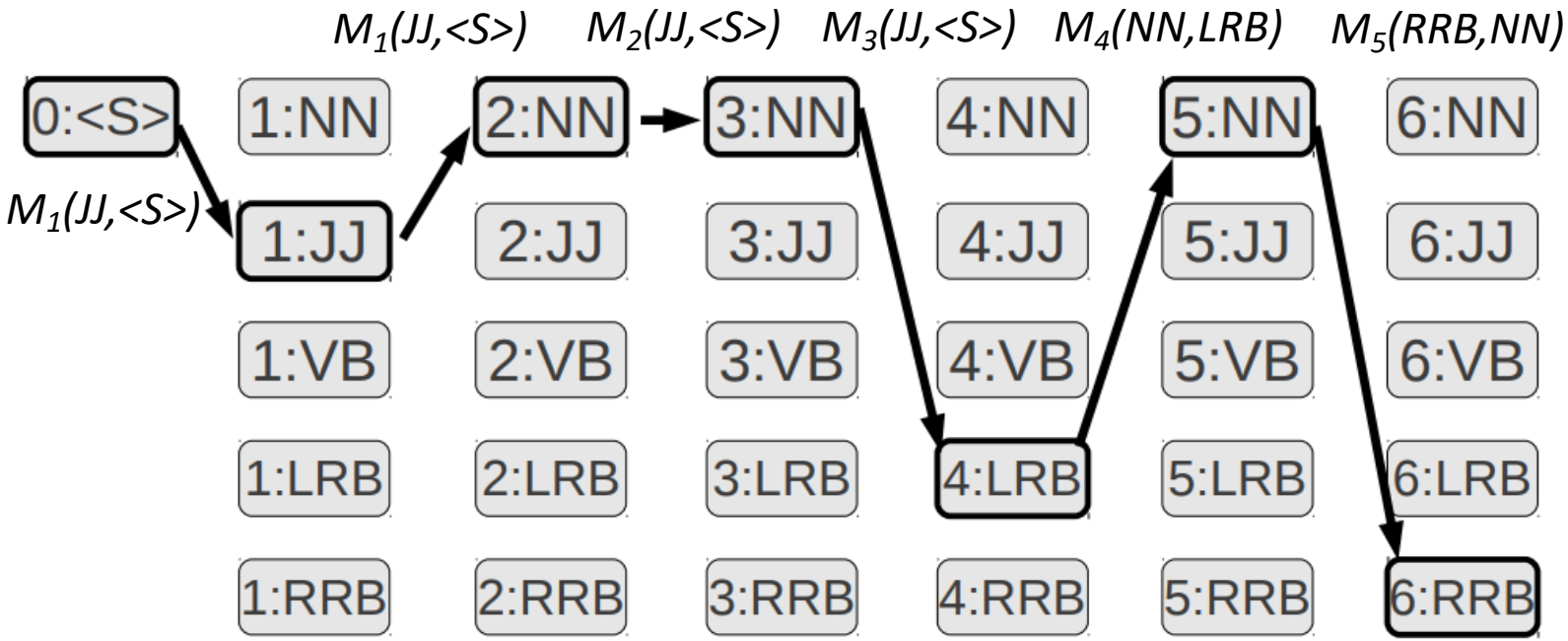
$$M_i(y, y') = exp(f(i, y', y, x)^T \cdot w)$$

$$\alpha_i(y) = \sum_{y_1,..,y_{i-1}=y} \prod_k M_k(y_{k-1}, y_k)$$

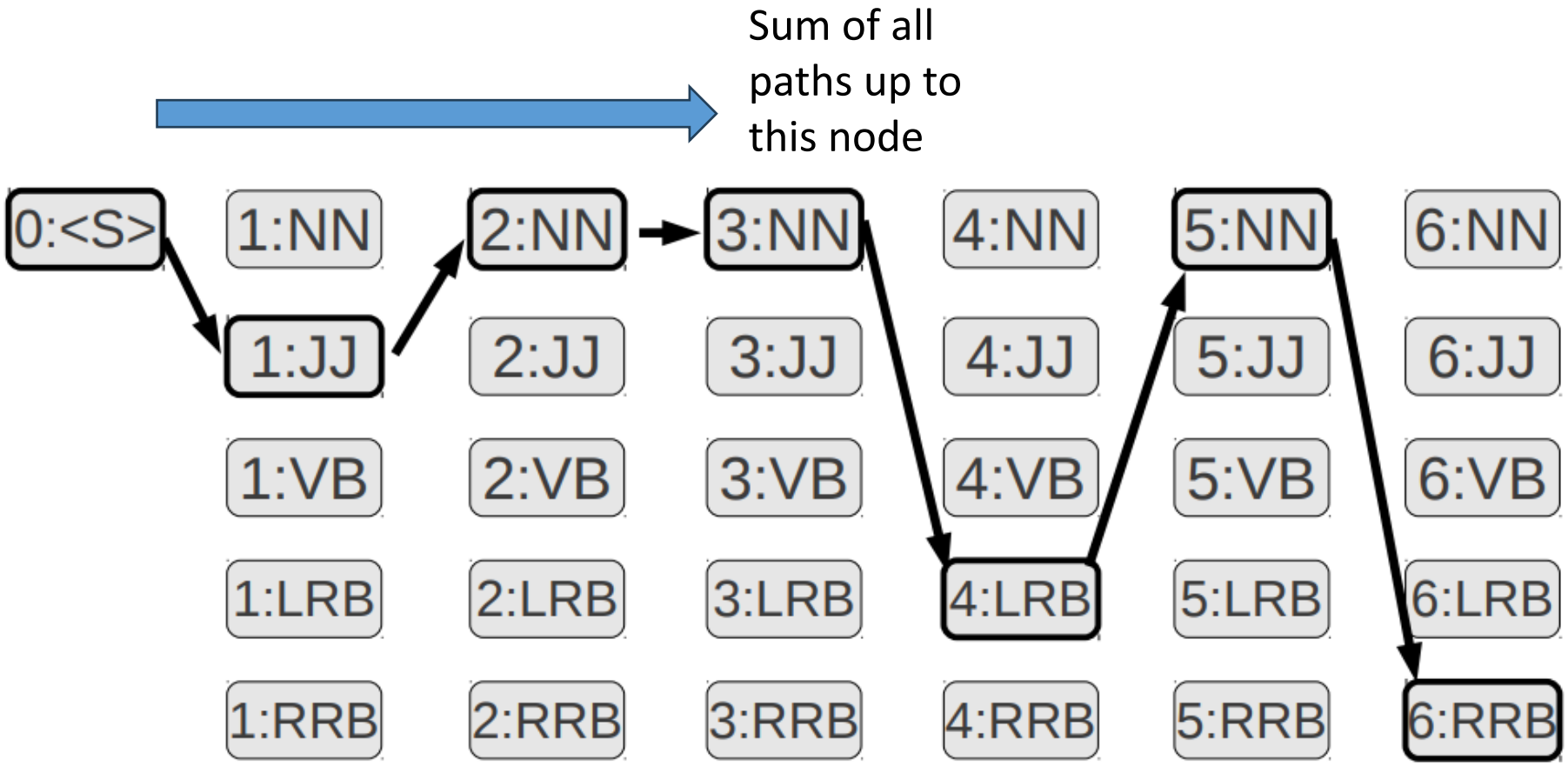
$$\beta_i(y) = \sum_{y_n,..,y_i=y} \prod_k M_k(y_k, y_{k+1})$$

Forward-Backward Algorithm in Viterbi Trellis

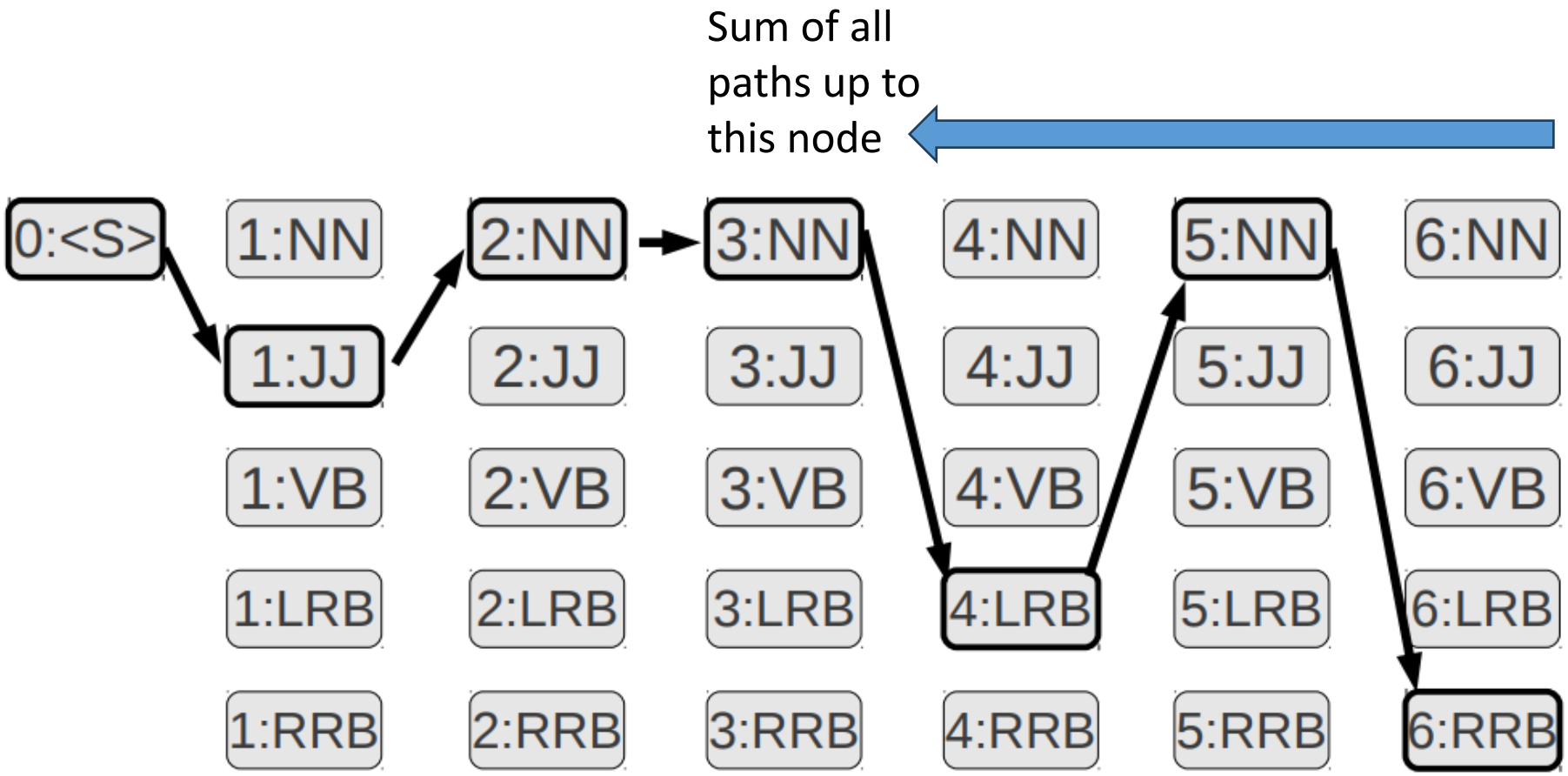
- The marginal $Pr(y_j, y_{j-1}|x^{(i)})$ is the sum over all paths that have the edge (y_{j-1}, y_j) between states *j-1* and *j*
- Edge weights between *y* and *y'* is $M_i(y, y')$



Forward Sum (α)



Backward Sum (β)



CRFs: Forward-Backward Algorithm

- The parameters can be computed using dynamic programming with these formulas (for *i*=1, computing α, same for β with *i*=n)

$$\alpha_i(y) = \sum_{y'} M_i(y', y)\alpha_{i-1}(y') \qquad \beta_i(y) = \sum_{y'} M_{i+1}(y, y')\beta_{i+1}(y')$$

- The marginal can now be computed as:

$$Pr(y_j, y_{j-1}|x^{(i)}) = \frac{M_j(y_j, y_{j-1}) \cdot \beta_j(y_j) \cdot \alpha_j(y_{j-1})}{Z(x^{(i)})}$$

CRFs: Forward-Backward Algorithm

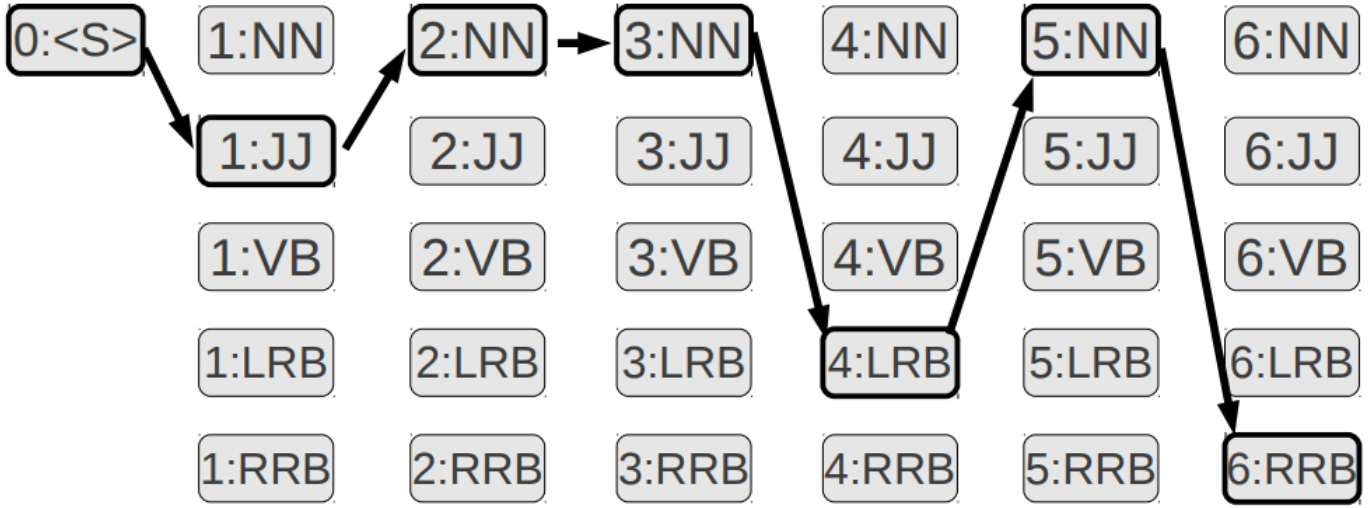
- Computing *Z(x)* can be done by summing the numerator over all possible values for y_j *and* y_{j-1}
  - A practical solution since we need to compute the marginals for every value of y_j and y_{j-1} anyway
- It can also be done directly by employing dynamic programming, or using the computed $\alpha_{n+1}(y)$:

$$Z(x) = \sum_{y} \alpha_{n+1}(y)$$

CRFs: Inference

- Inference again uses the Viterbi algorithm

- The same idea as with MEMMs. Edge weights between a pair of tags *y* and *y'* is $M_i(y, y')$



Some Accuracies on English Supervised POS Tagging

- Rough accuracies: (overall accuracy/unknown words)
  - Most frequent tag: ~90% / ~50%
  - Trigram HMM: ~95% / ~55%
  - TnT – Trigram HMM with better smoothing and handling of unknown words (Brants, 2000): 96.7% / 85.5%
  - Local MaxEnt: 96.8% / 86.8%
  - MEMM tagger: 96.8% / 86.9%
  - Structured Perceptron: 97.1% (we'll talk about this algorithm later in the course)
  - Cyclic tagger (multiplying two MEMMs) / CRFs: 97.2% / 89.0%
  - Inter-annotator agreement: ~98%

Domain Effects

- Accuracies degrade outside of domain
  - Up to triple error rate
  - Usually make the most errors on the things you care about in the domain (e.g. protein names)
- Open questions
  - How to effectively exploit unlabeled data from a new domain (what could we gain?)
  - How to best incorporate domain lexica in a principled way (e.g. UMLS specialist lexicon, ontologies)

Other Applications of CRFs

- CRFs have been used extensively in NLP, Vision and Computational Biology
- A few references: (#citations is a tricky measure, but the paper that introduced CRFs has about 13K of them..)
  - Named Entity Recognition (e.g., identifying protein names in biology papers)
  - Chinese Word Segmentation
  - Prediction of pitch accents
  - Semantic role labeling
  - Shallow Parsing
  - Syntactic Parsing

See section 2.7 in this CRF tutorial:
http://homepages.inf.ed.ac.uk/csutton/publications/crftut-fnt.pdf